

CEP I.1

Campus Network Link Design

Project guide and step-by-step roadmap

A plain-language guide to what we are building, why, and exactly how to build it, in order. Use it as the shared plan for the whole team.

Part	What it covers
A. Project brief	The problem, the solution, how it is built, key terms, who does what
B. Step-by-step plan	20 steps. Each explains why, what to do, what you should see and what file it produces
C. Wrap-up	Timeline, final checklist

How to read each step

Section	Meaning
WHY WE DO THIS	The purpose of the step
WHAT TO DO	The method, with worked examples
WHAT YOU SHOULD SEE	How to know you did it right
OUTPUT	The file or result you hand to the next step

Colour of the step header shows who owns it

Everyone	Python pair	Packet Tracer pair	Documentation
----------	-------------	--------------------	---------------

Part A: Project brief

1. The problem

A university needs to connect **six buildings**. The buildings are between 40 m and 1.8 km apart. Each connection (called a **link**) must carry a certain amount of data per second and must stay working for a certain share of the year.

Two links are special. One passes through a service corridor full of **electrical interference (EMI)**. Another crosses **open ground with no conduit**, so the cable has no protection. On top of this, the **budget is fixed** and too small to use fibre everywhere.

We must choose the cable type for every link, choose how data is written onto the cable, and **prove with calculations** that each link works.

Four things pull against each other

Factor	What happens	Why it matters
Distance	The signal gets weaker the longer the cable is	Copper stops working beyond about 100 m
Bandwidth	A cable can only carry signals up to a certain speed	Limits how fast data can go, and fibre loses bandwidth over distance
Reliability	Better uptime needs better parts or a spare link	Costs more money
Cost	Fibre and spare links are expensive	Budget is fixed, so we cannot simply buy the best everywhere

2. The solution we are building

We are building a **small engineering toolkit plus a network model**. It takes the description of the campus, tests every possible cable choice for every link, rejects the ones that fail, picks the cheapest one that works, and then proves the final network works in Packet Tracer.



The six parts of the finished project

Part	What it is	Built by
1. Scenario files	Three CSV tables: the links, the cable options, and settings such as the budget. Nothing is fixed inside the code, so changing a distance or price just means editing a table and running again.	Everyone
2. Link-budget engine	Python code that checks one link with one cable type: signal loss, noise, speed limits, uptime and cost.	Python pair
3. Selection engine	Python code that throws out options that fail any test (and records why), then picks the cheapest one that passes.	Python pair
4. Sensitivity engine	Python code that cuts the budget by 15% and finds which link breaks first.	Python pair
5. Network model	A Packet Tracer file with six buildings, devices, IP addresses and cables that match the chosen media. It proves every building can reach every other.	Packet Tracer pair
6. Report	The document that joins calculations, the plan and the simulation into one argument.	Documentation

The five questions every option must pass

For each link and each cable type, the engine asks five questions. An option stays only if the answer to all five is yes.

- **Does enough signal reach the other end?** (signal loss and power margin)
- **Is the signal clear above the noise?** (signal-to-noise ratio)
- **Can the cable carry the required speed?** (must fit under both speed limits, Shannon and Nyquist)
- **Is the uptime target met?** (availability, adding a spare link if needed)
- **Can we afford it?** (total cost within budget)

What the final plan looks like (example values)

This shows the shape of the result. Your real choices will come from your own data.

Link	Distance	Chosen medium	Spare link	Reason
L1	40 m	Cat6 copper	No	Short and cheap, passes every test
L2	90 m	Cat6 copper	No	Within the 100 m copper limit
L3	250 m	Armoured multimode fibre	No	Too far for copper; no conduit, so armoured cable
L4	600 m	Single-mode fibre	No	EMI rules out copper; multimode too short
L5	1.2 km	Single-mode fibre	Yes	99.99% uptime target needs a spare path

Link	Distance	Chosen medium	Spare link	Reason
L6	1.8 km	Single-mode fibre	Yes	Longest link, 99.99% target

The engine also produces a table of rejected options (for example: *L4 copper rejected because noise is too high in the EMI corridor*), a cost table showing the plan fits the budget, and a sensitivity result naming the link that fails first at a 15% cut.

3. How we build it

- **Fix the scenario.** Write the links, cable options and budget into CSV files (Steps 1 to 4).
- **Calculate.** Python computes loss, noise, speed limits, uptime and cost for every option (Steps 5 to 11).
- **Select.** Remove failing options, choose the cheapest that passes, record every rejection (Step 11).
- **Stress-test.** Cut the budget 15% and find the first failure (Step 12).
- **Check the code.** Confirm the results by hand and by trying every combination (Step 13).
- **Model the network.** Build it in Packet Tracer and test it (Steps 14 to 17).
- **Join and document.** Compare everything, then write the report (Steps 18 to 20).

Program layout

File	Job
load.py	Reads the three CSV files
physics.py	Signal loss, noise, SNR, Shannon, Nyquist
linecode.py	Line-code rules and spectrum plots
reliability.py	Uptime (availability) calculations
select.py	Filtering, cost, cheapest choice, rejected list
sensitivity.py	Budget cut, downgrade rule, sweep
main.py	Runs everything and saves results and plots

How the two halves meet

Python decides **what** to build. Packet Tracer proves it works **as a network**. They share one file: **results/media_plan.csv**. Packet Tracer cannot simulate signal loss or noise, so it only checks addressing, routing, reachability, speed and duplex. The report should say this clearly.

What this project is not

It is not a real installation plan (no cable-route drawings or labour schedule). It is not a physics simulator (the physics is in Python). It does not use advanced optimisation; it uses a simple rule that can be explained and checked.

4. Key terms in plain words

Term	Plain meaning
Attenuation	How much signal is lost along the cable, measured in dB
Noise	Unwanted signal that mixes with the real signal
SNR	Signal-to-noise ratio: how much stronger the signal is than the noise
Shannon capacity	The fastest data rate the noise level allows
Nyquist limit	The fastest data rate the cable bandwidth allows
Line coding	The way 1s and 0s are turned into electrical or light signals
Availability	The share of time a link is working
Duplex	Full: both directions at once. Half: one direction at a time. Simplex: one direction only
EMI	Electromagnetic interference from nearby electrical equipment

5. Who does what

Who	Owns
Python pair	Steps 5 to 13: all calculations, selection, sensitivity and checking
Packet Tracer pair	Steps 14 to 17: design, build and test the network
Documentation member	Step 19, and writing alongside every other step
Everyone	Steps 1 to 4, 18 and 20

Part B: Step-by-step plan

Follow the steps in order. Each step names who owns it.

Stage 1: Set up

Day 1, the whole team together. Finish all four steps before starting Stage 2.

Step 1: Define the six links

EVERYONE

WHY WE DO THIS	Every calculation starts from these numbers. Clear inputs give answers we can defend, so the scenario should include the difficult cases from the brief.																																			
WHAT TO DO	• Start from this table and change it to suit your campus: <table><tr><th>Link</th><th>Distance</th><th>Rate</th><th>Uptime</th><th>Special</th></tr><tr><td>L1</td><td>40 m</td><td>1 Gbps</td><td>99.9%</td><td></td></tr><tr><td>L2</td><td>90 m</td><td>1 Gbps</td><td>99.9%</td><td></td></tr><tr><td>L3</td><td>250 m</td><td>1 Gbps</td><td>99.95%</td><td>Open ground, no conduit</td></tr><tr><td>L4</td><td>600 m</td><td>1 Gbps</td><td>99.95%</td><td>EMI-heavy corridor</td></tr><tr><td>L5</td><td>1.2 km</td><td>1 Gbps</td><td>99.99%</td><td></td></tr><tr><td>L6</td><td>1.8 km</td><td>1 Gbps</td><td>99.99%</td><td>Farthest building</td></tr></table>	Link	Distance	Rate	Uptime	Special	L1	40 m	1 Gbps	99.9%		L2	90 m	1 Gbps	99.9%		L3	250 m	1 Gbps	99.95%	Open ground, no conduit	L4	600 m	1 Gbps	99.95%	EMI-heavy corridor	L5	1.2 km	1 Gbps	99.99%		L6	1.8 km	1 Gbps	99.99%	Farthest building
Link	Distance	Rate	Uptime	Special																																
L1	40 m	1 Gbps	99.9%																																	
L2	90 m	1 Gbps	99.9%																																	
L3	250 m	1 Gbps	99.95%	Open ground, no conduit																																
L4	600 m	1 Gbps	99.95%	EMI-heavy corridor																																
L5	1.2 km	1 Gbps	99.99%																																	
L6	1.8 km	1 Gbps	99.99%	Farthest building																																
WHAT YOU SHOULD SEE	A mix of short links (copper may work), middle links (copper or fibre) and long links (fibre needed).																																			
OUTPUT	data/links.csv with columns: link_id, bldg_a, bldg_b, length_m, target_mbps, avail_target, emi_heavy, has_conduit.																																			

Step 2: Build the media catalogue

EVERYONE

WHY WE DO THIS	The program can only choose from options we list. Real datasheet numbers make the result believable.
WHAT TO DO	- Pick four options: Cat6 copper, multimode fibre (OM3), single-mode fibre, and optionally a radio link. - For each one, find: signal loss per km, bandwidth, maximum distance, cost per metre, install cost per metre, fixed cost (connectors, transceivers), MTBF and MTTR. - Write the source beside every number.
WHAT YOU SHOULD SEE	Copper is cheap but short. Multimode fibre sits in the middle. Single-mode goes furthest but costs most.

OUTPUT	data/media.csv with a source column.
--------	---

Step 3: Set the budget and assumptions		EVERYONE
WHY WE DO THIS	The brief says the budget cannot pay for fibre everywhere. That limit forces a real trade-off. Reviewers will question every assumption, so write them down.	
WHAT TO DO	• Price the "all single-mode" design and the "cheapest possible" design. Set the budget between the two. • List the assumptions: two transceivers per link; a spare link costs double and fails independently; which noise sources are counted.	
WHAT YOU SHOULD SEE	An all-fibre design is too expensive, but a mixed design fits.	
OUTPUT	data/params.csv (budget, 15% cut, safety factor 1.5, noise figure, connector loss, minimum margin) and a written assumptions list.	

Step 4: Set up the shared workspace		EVERYONE
WHY WE DO THIS	Two pairs working at once will overwrite each other unless the structure is agreed first.	
WHAT TO DO	• Create one shared folder or repository with: data, results, code, packet_tracer, report. • Agree one rule: the only file passed from Python to Packet Tracer is results/media_plan.csv.	
WHAT YOU SHOULD SEE	Everyone can open the files and run the code.	
OUTPUT	A shared workspace.	

Stage 2: Calculations (Python pair)

Steps 5 to 13. Each step adds columns to one results table.

Step 5: Load the data and build the option table		PYTHON PAIR
WHY WE DO THIS	One table with a row for every link and cable pair keeps every later result traceable.	
WHAT TO DO	• Read the three CSV files with pandas. • Combine every link with every cable type.	
WHAT YOU SHOULD SEE	6 links x 4 media = 24 rows.	
OUTPUT	results/options.csv	

Step 6: Signal loss and power margin		PYTHON PAIR
WHY WE DO THIS	Attenuation is signal lost along the cable. If too much is lost, the receiver hears nothing. This rules copper out on long links and is the first part of Task 1.	
WHAT TO DO	Copper: loss = dB per 100 m x length / 100. Example: 90 m of Cat6 at 100 MHz loses $22 \times 0.9 = \mathbf{19.8 \text{ dB}}$. Copper is not allowed beyond 100 m. Fibre: received power = transmit power - fibre loss - connector loss. L6, 1.8 km single-mode: -9 - 0.63 - 1.0 = -10.6 dBm The receiver needs -20 dBm, so the margin is 9.4 dB. Multimode fails on L6 because Gigabit multimode reaches only about 550 m. Radio: free-space loss = $20 \log_{10}(\text{km}) + 20 \log_{10}(\text{MHz}) + 32.44$. - Keep only options whose margin is at least the minimum in params.csv.	
WHAT YOU SHOULD SEE	Copper passes only the short links. Fibre passes everywhere. Multimode drops out on the long links.	
OUTPUT	Columns loss_db , margin_db , pass_power .	

Step 7: Noise and signal-to-noise ratio (SNR)		PYTHON PAIR
WHY WE DO THIS	A strong signal still fails if the noise is stronger. SNR tells us how clean the signal is. The EMI corridor shows up here.	
WHAT TO DO	Thermal noise (dBm) = $-174 + 10 \log_{10}(\text{bandwidth}) + \text{noise figure}$ - Add crosstalk noise and EMI noise. Convert each to milliwatts first, add them, then convert back. Never add dB values directly. - Give the EMI link a much larger EMI term. SNR = received power - total noise - Skip fibre. It is immune to EMI, and its test is the power margin from Step 6.	
WHAT YOU SHOULD SEE	Copper on the EMI link has a clearly lower SNR than copper elsewhere.	
OUTPUT	Columns noise_dbm , snr_db .	

Step 8: Shannon and Nyquist speed limits		PYTHON PAIR
WHY WE DO THIS	There are two speed limits. Shannon is set by noise. Nyquist is set by bandwidth. A link works only if the target rate fits under both. This is the feasibility proof.	

WHAT TO DO	Shannon: $C = B \times \log_2(1 + \text{SNR})$ (SNR as a ratio) Example: $B = 100 \text{ MHz}$ and $\text{SNR} = 30 \text{ dB}$ (ratio 1000) gives about 997 Mbps. The target of $1 \text{ Gbps} \times \text{safety factor } 1.5 = 1.5 \text{ Gbps}$, so this option fails. Nyquist: $\text{max rate} = 2 \times B \times \log_2(M)$ ($M = \text{signal levels}$; 2 for binary) - Fibre bandwidth shrinks with distance: $B = (\text{MHz} \cdot \text{km}) / \text{length in km}$. - Require both limits to pass.
WHAT YOU SHOULD SEE	Some copper options pass the power test but fail on rate. This is why both checks are needed.
OUTPUT	Columns shannon_mbps , nyquist_mbps , pass_rate .

Step 9: Choose the line code			PYTHON PAIR															
WHY WE DO THIS	The line code decides how much bandwidth the signal uses and whether the receiver can keep its clock in sync. Task 1 asks us to justify the choice.																	
WHAT TO DO	<table><thead><tr><th>Code</th><th>Bandwidth needed</th><th>Keeps the clock in sync?</th></tr></thead><tbody><tr><td>NRZ</td><td>About $R/2$</td><td>Poorly. Long runs of the same bit lose sync</td></tr><tr><td>RZ</td><td>About R</td><td>Better, but uses more bandwidth</td></tr><tr><td>Manchester</td><td>About R</td><td>Yes. Every bit has a transition (50% efficient)</td></tr><tr><td>Block codes (4B/5B, 8b/10b, 64b/66b)</td><td>Low per bit</td><td>Yes. Run length is limited</td></tr></tbody></table>	Code	Bandwidth needed	Keeps the clock in sync?	NRZ	About $R/2$	Poorly. Long runs of the same bit lose sync	RZ	About R	Better, but uses more bandwidth	Manchester	About R	Yes. Every bit has a transition (50% efficient)	Block codes (4B/5B, 8b/10b, 64b/66b)	Low per bit	Yes. Run length is limited		
	Code	Bandwidth needed	Keeps the clock in sync?															
	NRZ	About $R/2$	Poorly. Long runs of the same bit lose sync															
	RZ	About R	Better, but uses more bandwidth															
	Manchester	About R	Yes. Every bit has a transition (50% efficient)															
	Block codes (4B/5B, 8b/10b, 64b/66b)	Low per bit	Yes. Run length is limited															
- Choose the code that uses the least bandwidth and still keeps sync. Use Manchester only if the channel bandwidth is at least equal to the data rate; otherwise use a block code. - Plot the power spectrum of each code for a random bit stream with a fixed seed.																		
WHAT YOU SHOULD SEE	One justification sentence per link, plus a plot showing NRZ is narrowest and Manchester is widest.																	
OUTPUT	Column <code>line_code</code> , one spectrum figure and the justification text.																	

Step 10: Availability (uptime)		PYTHON PAIR
WHY WE DO THIS	Each link has an uptime target. This shows whether one link is enough or a spare is needed.	

WHAT TO DO	$A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$ - Link uptime = transceiver x cable x transceiver. - If below target, test two parallel links: $A = 1 - (1 - A_{\text{link}})^2$ - Convert to downtime: 99.9% is about 8.8 hours a year; 99.99% is about 53 minutes.
WHAT YOU SHOULD SEE	The 99.99% links need a spare or better parts. The 99.9% links pass alone.
OUTPUT	Columns avail , redundancy_needed .

Step 11: Cost and the media plan		PYTHON PAIR
WHY WE DO THIS	This is the decision step and answers Task 2. It combines every earlier test with the budget.	
WHAT TO DO	$\text{cost} = \text{fixed} + (\text{cost per m} + \text{install per m}) \times \text{length}$ - Double the cost for a spare link. Add trenching or armoured cable for the no-conduit link. - Remove options that failed Steps 6, 8 or 10, then choose the cheapest one left for each link. - If the total is over budget, downgrade the links that save the most money for the least harm. Record every rejected option and the test it failed.	
WHAT YOU SHOULD SEE	A mixed plan inside the budget, plus a table explaining every rejection.	
OUTPUT	results/media_plan.csv (link, media, rate, duplex, line code) and results/rejected.csv .	

Step 12: Sensitivity analysis: 15% budget cut		PYTHON PAIR
WHY WE DO THIS	Answers Task 4. It shows which link is weakest when money gets tighter.	
WHAT TO DO	- New budget = 0.85 x budget. Run the downgrade rule again. - Score each link's shortfall in length, rate and availability. The first link with a shortfall is the answer. - Repeat at 0, 5, 10 ... 30% and plot the results.	
WHAT YOU SHOULD SEE	One named link with a clear reason, plus a chart of shortfalls against budget cut.	
OUTPUT	results/sensitivity.csv and one plot.	

Step 13: Check the code		PYTHON PAIR
WHY WE DO THIS	Reviewers will test the numbers. Finding our own mistakes first protects everything after this step.	
WHAT TO DO	• Check two links by hand or in a spreadsheet. • Try every combination of media (it is a small search) to confirm the plan is truly the cheapest that works. • Test the dB to milliwatt conversion.	
WHAT YOU SHOULD SEE	Hand results and code agree, and the cheapest-plan check matches.	
OUTPUT	A short verification table.	

Stage 2b: Network model (Packet Tracer pair)

Steps 14 to 17. Start at the same time as Stage 2.

Step 14: Plan the network on paper		PACKET TRACER PAIR
WHY WE DO THIS	IP addressing mistakes are hard to fix later, so plan first.	
WHAT TO DO	• Each building gets one router (or layer-3 switch), one LAN switch and a few PCs. • Addressing: one /30 subnet per building-to-building link and one LAN subnet per building, sized for its hosts (VLSM). • Write the address table before opening Packet Tracer.	
WHAT YOU SHOULD SEE	A complete table with no overlapping subnets.	
OUTPUT	Address table and topology sketch.	

Step 15: Build the skeleton		PACKET TRACER PAIR
WHY WE DO THIS	A working network early on means that later only the cables change.	
WHAT TO DO	• Place devices and connect them with placeholder cables. • Set IP addresses and add static routes (or OSPF). • Test with ping.	
WHAT YOU SHOULD SEE	Every PC can ping every other PC.	
OUTPUT	A working .pkt file.	

Step 16: Apply the media plan

PACKET TRACER PAIR

WHY WE DO THIS	This makes the simulation match the engineered design.
WHAT TO DO	- When media_plan.csv is final, swap the cables: copper straight-through, or fibre with fibre modules at both ends. - Set speed and duplex full on each interface. - Note: Packet Tracer does not simulate signal loss or noise. The link budget is proven in Python; Packet Tracer proves the network works.
WHAT YOU SHOULD SEE	The topology matches the plan and all links are up.
OUTPUT	Final .pkt file.

Step 17: Validate

PACKET TRACER PAIR

WHY WE DO THIS	Task 3 asks for proof of reachability and duplex settings.
WHAT TO DO	- Run ping and traceroute between every pair of buildings. - Run show ip route and show interfaces (speed, duplex, errors). - Shut one link to show traffic rerouting where a spare exists. - Duplex: use full duplex on all switched links. Half duplex only applies to hubs. Packet Tracer has no true simplex mode, so state simplex as an assumption and explain why data links do not use it.
WHAT YOU SHOULD SEE	All tests pass and duplex matches on both ends of every link.
OUTPUT	A results table with screenshots.

Stage 3: Bring it together

Steps 18 to 20. The whole team, with documentation running throughout.

Step 18: Integrate

EVERYONE

WHY WE DO THIS	Find mismatches between the calculations and the simulation before writing the report.
WHAT TO DO	- Compare the .pkt file with media_plan.csv link by link. - Sanity check: the EMI link avoids unshielded copper, long links use single-mode, total cost is within budget.
WHAT YOU SHOULD SEE	No mismatches. The design is frozen.
OUTPUT	Frozen design.

Step 19: Write the report

DOCUMENTATION

WHY WE DO THIS	The report is what gets graded. It must show the reasoning, not only the result.														
WHAT TO DO	- Start in Stage 1 and write each section as its step finishes. <table><tr><th>Report section</th><th>Comes from</th></tr><tr><td>Scenario and assumptions</td><td>Steps 1 to 3</td></tr><tr><td>Method and formulas</td><td>Steps 6 to 10</td></tr><tr><td>Per-link results table</td><td>Steps 6 to 10</td></tr><tr><td>Media plan and rejected options</td><td>Step 11</td></tr><tr><td>Packet Tracer design and tests</td><td>Steps 14 to 17</td></tr><tr><td>Sensitivity and conclusion</td><td>Step 12</td></tr></table> - Generate tables straight from the CSV files so the numbers always match the code.	Report section	Comes from	Scenario and assumptions	Steps 1 to 3	Method and formulas	Steps 6 to 10	Per-link results table	Steps 6 to 10	Media plan and rejected options	Step 11	Packet Tracer design and tests	Steps 14 to 17	Sensitivity and conclusion	Step 12
Report section	Comes from														
Scenario and assumptions	Steps 1 to 3														
Method and formulas	Steps 6 to 10														
Per-link results table	Steps 6 to 10														
Media plan and rejected options	Step 11														
Packet Tracer design and tests	Steps 14 to 17														
Sensitivity and conclusion	Step 12														
WHAT YOU SHOULD SEE	Every claim points to a table, plot or screenshot.														
OUTPUT	The final report.														

Step 20: Review and rehearse

EVERYONE

WHY WE DO THIS	Reviewers may ask any member about any part.
WHAT TO DO	- Swap sections between the two pairs and read them critically. - Check every number in the report against the code output. - Each member practises explaining all four tasks.
WHAT YOU SHOULD SEE	No inconsistencies, and everyone can defend the whole project.
OUTPUT	A project ready to submit.

Part C: Wrap-up

Suggested timeline

Phase	Share of time
Stage 1: Set up (Steps 1 to 4)	5%
Stages 2 and 2b: Calculations and network (Steps 5 to 17)	55%
Integration (Step 18)	10%
Report and review (Steps 19 to 20)	30%

Documentation runs alongside every phase. Scale the percentages to your deadline.

Final checklist

- links.csv, media.csv and params.csv are complete, with sources
- Per-link table: loss, noise, SNR, Shannon, Nyquist, line code, uptime, cost
- Media plan and the table of rejected options
- Packet Tracer file with ping and duplex evidence
- Answer to the 15% budget cut, with the sweep plot
- Report, code and Packet Tracer file all agree